

UNIVERSITATEA TEHNICĂ "GHEORGHE ASACHI" DIN IAȘI  
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE  
MASTER: SECURITATEA SPAȚIULUI CIBERNETIC

# Protocol de autentificare OAuth cu Schnorr ZKP: Securitate fără partajare de secrete

---

Autor: Matei Rareș

Coordonator: Ș.I.dr.inf. Silviu Pavăl

# SUMAR

---

Introducere

---

Tehnologii

---

Arhitectură

---

Schnorr

---

Inregistrarea & Autentificare

---

Proprietati de Securitate

---

Validare si rezultate

---

Evaluarea performantei (comparatii)

---

Directii de dezvoltare

---

Concluzii

---

Bibliografie

---

# INTRODUCERE

---

Autentificarea OAuth clasica: secret partajat transmis către server

HTTPS: reduce interceptarea, nu elimină vulnerabilitățile structurale (breșe DB, reutilizare parole, SNDL)

**Solutie:** Protocol de autentificare fără transmiterea parolei către server.  
, Schnorr

Mapare ZKP peste OAuth: angajament → inițiere cerere, provocare → nonce sesiune, răspuns → autentificare client

Referință: 3 fluxuri OAuth clasice + evaluare comparativă protocoale ZKP (SRP-6a, zk-SNARK)

# TEHNOLOGII



Javascript, HTML, CSS

Compatibilitate si  
standardizare



Python ( Flask, Locust )

Limbaj flexibil pentru  
îmbinarea mai multor  
biblioteci, iterare rapida

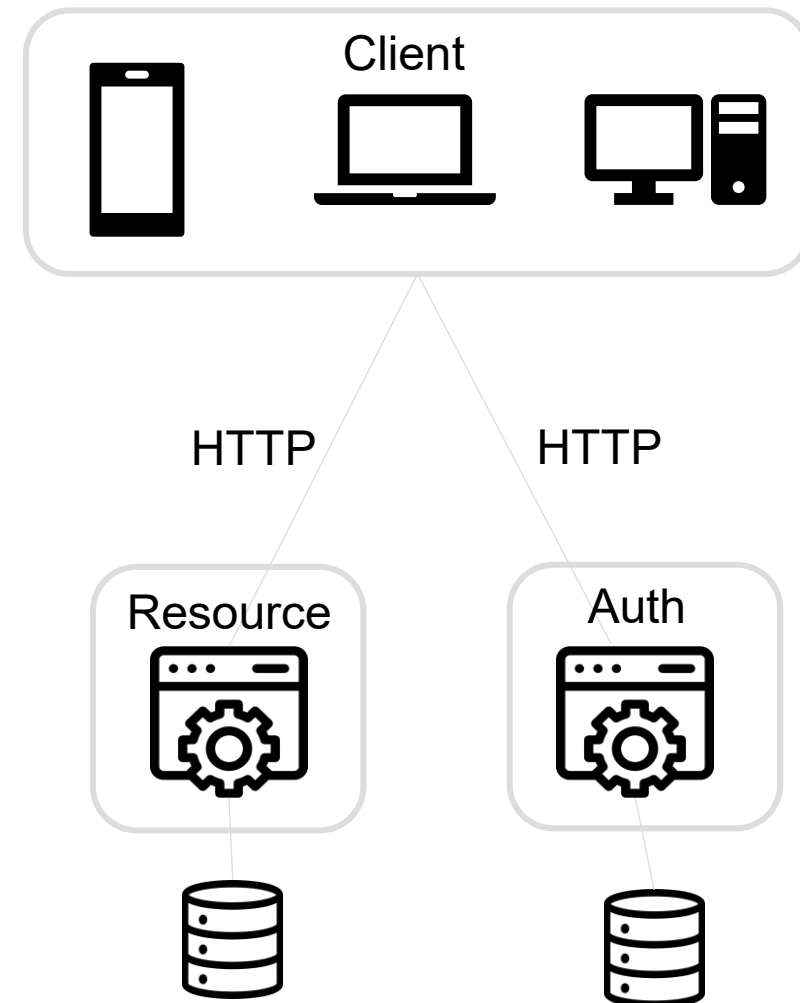
# ARHITECTURA environment

**Prezentare:** interfață web statică, calcule ZKP locale, monitor rețea

**Aplicație (Flask):** validare subgrup (is\_subgroup\_member),  
emitere JWT, legare canal (Session Binding)

**Date (SQLite):** User (secret\_y), AuthToken, PersoData

**\*\*Diagramă arhitectură 3 niveluri: Client ↔ Flask ↔ SQLite\*\***



# Schnorr interactive

---

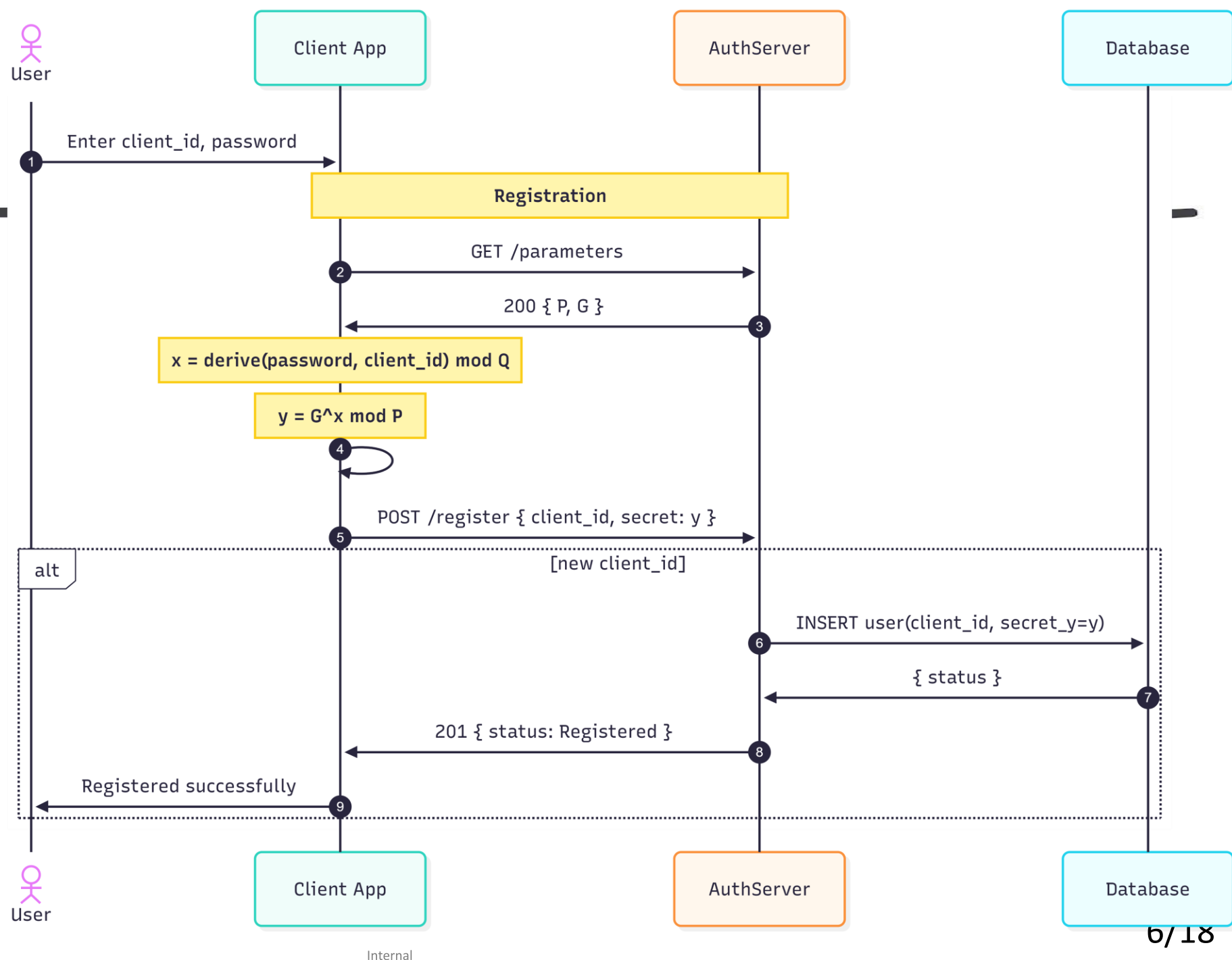
Fundament: problema logaritmului discret (DLP)  
prim sigur  $P = 2Q + 1$ ,  $G = h^2 \bmod P$ ,  $G \neq 1$ ,  $G^Q \bmod P = 1$ ,  $h \in [0, P]$

1. Chei:  $y = G^x \bmod P$
2. Angajament:  $t = G^r \bmod P$
3. Provocare:  $c \leftarrow \text{aleatoriu din } \mathbb{Z}_Q$
4. Răspuns:  $s = (r + c * x) \bmod Q$
5. Verificare:  $G^s \equiv t * y^c \pmod{P}$

Corectitudine:  $G^s = G^{(r+cx)} = G^r * G^{(cx)} = t * y^c \bmod P$

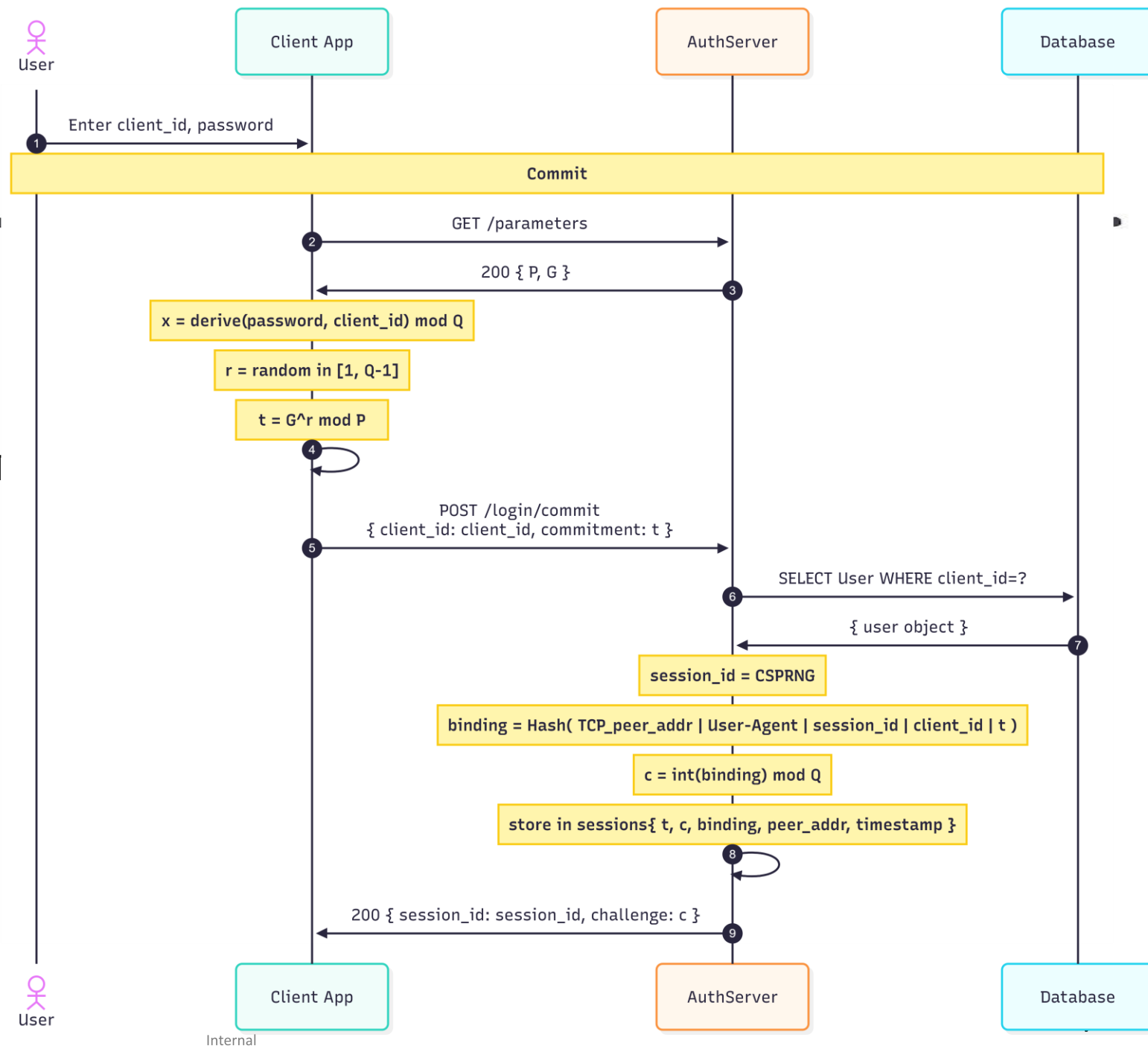
# Inregistrarea

- Client: derivare locală  $x$  din parolă (script + salt unic)
- Calcul:  $y = G^x \bmod P$
- Transmis  $\rightarrow$  server: exclusiv  $y$
- Server: validare  $y^Q \bmod P = 1$ ,  $y \notin \{0, 1, -1, P-1\} \rightarrow$  stocare
- Parolă + cheie privată: nu părăsesc dispozitivul
- Compromitere DB:  $y \rightarrow x = \text{DLP}$  pe 2048 biți (nefezabil)



# Autentificare commit

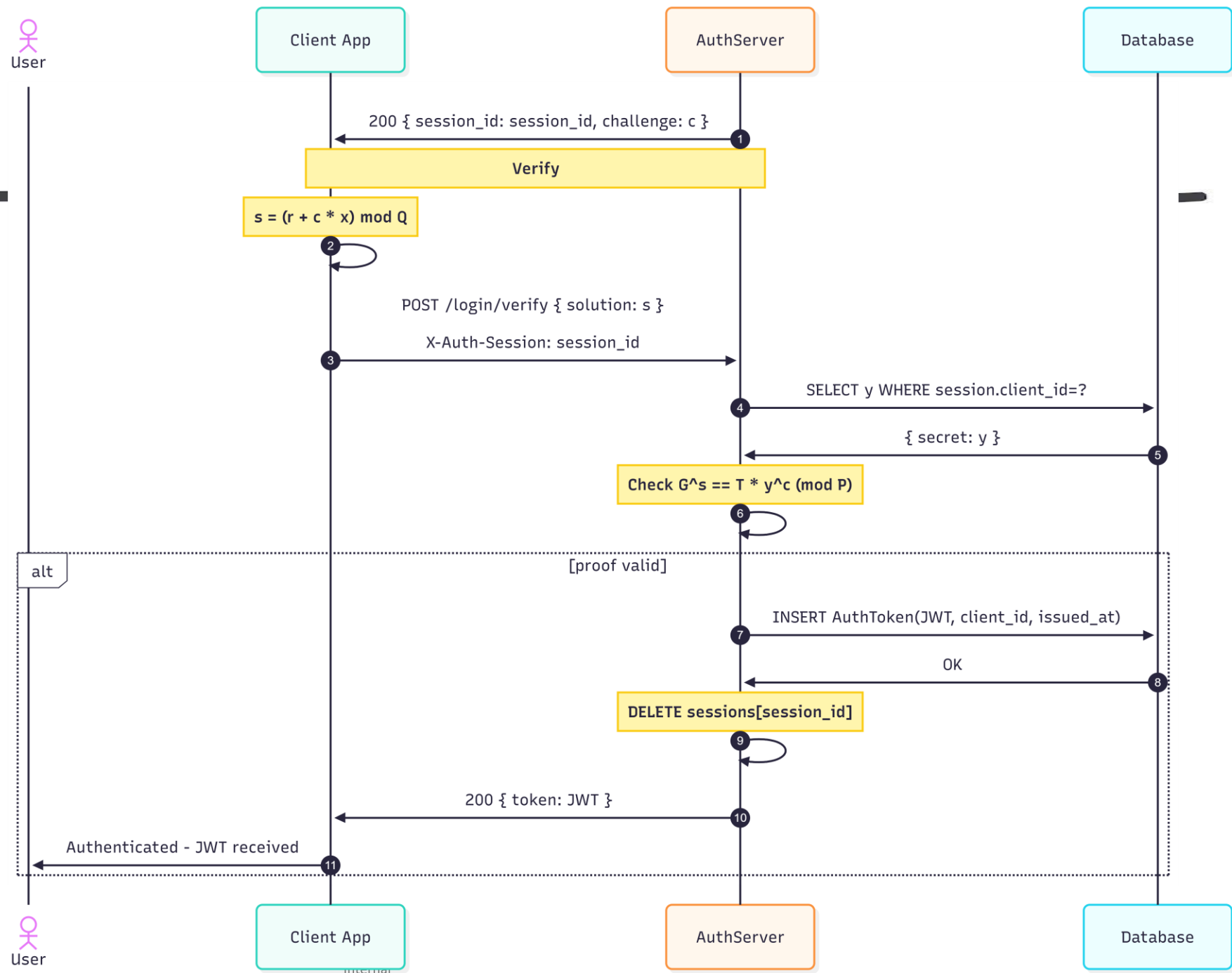
Client  $\rightarrow$  t, client\_id (r aleatoriu,  
window.crypto.getRandomValues)  
Server  $\rightarrow$  c, session\_id (validare subgrup t, 1  
Client  $\rightarrow$  s = (r + c \* x) mod Q  
Server:  $G^s \equiv t * y^c \pmod{P} \rightarrow$  JWT (chan  
Sesiune invalidată imediat după consum





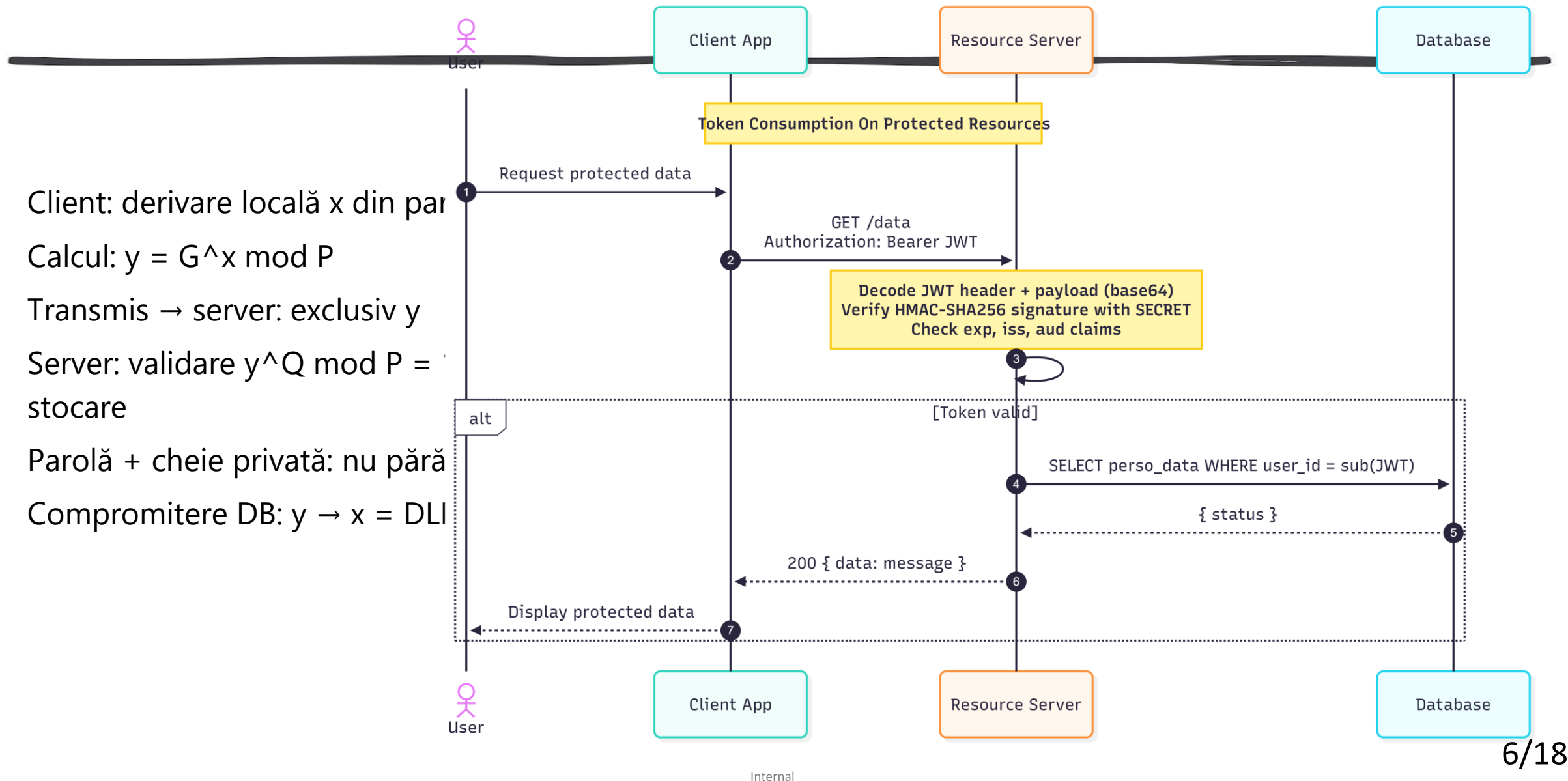
# Autentificare verify

Client  $\rightarrow$  t, client\_id (r aleatoriu, window.crypto.getRandomValue s)  
Server  $\rightarrow$  c, session\_id (validare subgrup t, TTL = 5s, mutex)  
Client  $\rightarrow$   $s = (r + c * x) \bmod Q$   
Server:  $G^s \equiv t * y^c \pmod{P} \rightarrow$  JWT (channel binding)  
Sesiune invalidată imediat după consum



# Consum token

- Client: derivare locală  $x$  din parolă
- Calcul:  $y = G^x \bmod P$
- Transmis  $\rightarrow$  server: exclusiv  $y$
- Server: validare  $y^Q \bmod P = 1$
- Stocare
- Parolă + cheie privată: nu părăsesc serverul
- Compromitere DB:  $y \rightarrow x = \text{DLI}$



# Proprietăți de securitate

---

**SNDL:** {t, c, s} efemere, r netransmis → reziliență interceptare

**Zero secret partajat:** server stochează doar y

**Anti-replay:** c unic per sesiune, invalidare post-consum

**Session Binding:** JWT ancorat la contextul HTTP al sesiunii ZKP

| No. | Time         | Source    | Destination | Protocol  | Length | Info                               |
|-----|--------------|-----------|-------------|-----------|--------|------------------------------------|
| ... | 9.883685800  | 127.0.0.1 | 127.0.0.1   | HTTP      | 537    | OPTIONS /parameters HTTP/1.1       |
| ... | 9.887015100  | 127.0.0.1 | 127.0.0.1   | HTTP      | 864    | HTTP/1.1 200 OK                    |
| ... | 10.150213900 | 127.0.0.1 | 127.0.0.1   | HTTP      | 660    | GET /parameters HTTP/1.1           |
| ... | 10.152405500 | 127.0.0.1 | 127.0.0.1   | HTTP/J... | 225    | HTTP/1.1 200 OK , JSON (applic...  |
| ... | 10.200326300 | 127.0.0.1 | 127.0.0.1   | HTTP      | 553    | OPTIONS /login/commit HTTP/1.1     |
| ... | 10.201559900 | 127.0.0.1 | 127.0.0.1   | HTTP      | 873    | HTTP/1.1 200 OK                    |
| ... | 10.468373500 | 127.0.0.1 | 127.0.0.1   | HTTP/J... | 888    | POST /login/commit HTTP/1.1 , J... |
| ... | 10.477661700 | 127.0.0.1 | 127.0.0.1   | HTTP/J... | 208    | HTTP/1.1 200 OK , JSON (applic...  |
| ... | 10.517033600 | 127.0.0.1 | 127.0.0.1   | HTTP      | 568    | OPTIONS /login/verify HTTP/1.1     |
| ... | 10.518286600 | 127.0.0.1 | 127.0.0.1   | HTTP      | 889    | HTTP/1.1 200 OK                    |

▶ Frame 431: 888 bytes on wire (7104 bits), 888 bytes captured (7104 bits) on interface \Device\NPF\_{Loopback}

▶ Null/Loopback

▶ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

▶ Transmission Control Protocol, Src Port: 55402, Dst Port: 5000, Seq: 1, Ack: 1, Len: 844

▶ Hypertext Transfer Protocol

▶ JavaScript Object Notation: application/json

▼ Object

- Member: client\_id
  - [Path with value: /client\_id:a]
  - [Member with value: client\_id:a]
  - String value: a
  - Key: client\_id
  - [Path: /client\_id]
- Member: commitment\_t
  - [Path with value: /commitment\_t:60954298276861417416912049145112403047201145644934292144036591470150963811728801]
  - [Member with value: commitment\_t:60954298276861417416912049145112403047201145644934292144036591470150963811728801]
  - String value: 60954298276861417416912049145112403047201145644934292144036591470150963811728801
  - Key: commitment\_t
  - [Path: /commitment\_t]

**\*\*Captură Wireshark: zero parolă/derivate în trafic HTTP\*\***

**\*\*Grafic escaladare exponențială forță brută (biți vs. timp)\*\***

# Validare si testare

**49 teste pytest, 100% succes**

| FORȚĂ BRUTĂ DLP | BIȚI                       | REZULTAT                |
|-----------------|----------------------------|-------------------------|
| 8 biți          | 83 candidați               | Rezolvat: 26 încercări  |
| 32 biți         | $\sim 1,07 \cdot 10^9$     | 744M încercări (319s)   |
| 64 biți         | $\sim 4,6 \cdot 10^{18}$   | Timeout 1200s           |
| 2048 biți       | $\sim 5,87 \cdot 10^{153}$ | Nefezabil computațional |

| CATEGORIE  | NR. | ACOPERIRE                                                                                                                                    |
|------------|-----|----------------------------------------------------------------------------------------------------------------------------------------------|
| Pozitive   | 5   | Absență parolă DB (plain/hash), flux complet, multi-user simultan                                                                            |
| Negative   | 5   | Parolă greșită (y furat, fără x), replay, expirare sesiune, commit dublu (409), înregistrare duplicată                                       |
| Limită     | 31  | $s \notin \mathbb{Z}_Q$ , $y/t \in \{0,1,-1,P-1\}$ , 10 fire concurente (mutex), tipuri neconforme (float/string/null), lipsă X-Auth-Session |
| Securitate | 8   | Simulator Schnorr (commitment binding), $t=y$ , forgery fără x, off-by-one, izolare sesiuni, unicitate, y furat                              |

| ATAC SIMULAT                                   | REZULTAT                                     |
|------------------------------------------------|----------------------------------------------|
| Compromitere DB: $s = (r + c \cdot y) \bmod Q$ | HTTP 401: $y \neq x$ în spațiul exponenților |
| Coliziune: 10.000 cereri                       | 0 coliziuni c + session_id                   |
| MitM parametri slabi: P=23, Q=11, G=4          | HTTP 422: y/t respinse (subgrup invalid)     |
| Replay cross-sesiune                           | HTTP 401: $G^s s_1 \neq t_2^* y^{c_2}$       |
| Brute-force session_id (256 biți)              | 500 tentative → toate HTTP 404               |

# Evaluare performanta

---

| FLUX                  | MEDIE (MS) | MIN   | MAX   | P95   |
|-----------------------|------------|-------|-------|-------|
| ZKP (commit + verify) | 64,76      | 59,24 | 85,98 | 78,71 |
| OAuth2 PKCE           | 2,23       | 1,81  | 22,21 | 1,58  |
| OAuth2 Simple         | 2,19       | 1,82  | 14,23 | 2,72  |
| Authlib PKCE          | 0,79       | 0,71  | 1,68  | 0,95  |

## Debit concurent (Locust, 8 fire, 60s)

| UTILIZATORI | ZKP (RPS) | PKCE   | SIMPLE | AUTHLIB |
|-------------|-----------|--------|--------|---------|
| 10          | 19,61     | 464,82 | 440,26 | 489,10  |
| 50          | 18,10     | 465,68 | 463,45 | 481,65  |
| 100         | 17,47     | 458,06 | 467,56 | 505,24  |

# Evaluare performanta oauth vs zkp

---

| METRICĂ           | ZKP SCHNORR | OAUTH 2.0 (AUTHLIB) |
|-------------------|-------------|---------------------|
| Latență medie     | 64,76 ms    | 0,79 ms             |
| Debit (100 users) | 17,47 RPS   | 505,24 RPS          |
| Raport            | 1           | :67                 |
| Parolă în rețea   | NU          | DA                  |
| Reziliență SNDL   | DA          | NU                  |

Compromis justificat: eliminare completă risc exfiltrare  
parolă, debit ZKP stabil indiferent de concurență

# Evaluare performanta srp si zk-snark

Latență (100 iterații)

| PROTOCOL               | ÎNREG. (MS) | AUTENTIF. (MS) | TOTAL (MS) |
|------------------------|-------------|----------------|------------|
| pysnark (zk-SNARK)     | 0,048       | 1,872          | 1,920      |
| Schnorr-EC-C-Lib       | 0,071       | 9,447          | 9,510      |
| Schnorr-EC (secp256r1) | 4,005       | 34,093         | 38,090     |
| SRP-6a (RFC 5054)      | 1,204       | 41,061         | 42,265     |
| Schnorr-Pow /MODP      | 15,783      | 139,833        | 155,911    |

| PROPRIETATE           | SRP-6A   | ZK-SNARK | SCHNORR-EC |
|-----------------------|----------|----------|------------|
| Autentificare mutuală | DA       | NU       | DA         |
| Parolă netransmisă    | DA       | DA       | DA         |
| Rezistență breșe DB   | DA       | N/A      | NU         |
| Securitate            | ~112 b   | variabil | ~128 b     |
| Standardizat          | RFC 5054 | NU       | NU         |
| Trusted setup         | NU       | DA       | NU         |

# Directii de dezvoltare

---

Derivare secrete: Argon2id / scrypt  
OWASP

Migrare curbe eliptice (secp256r1):  
latență redusă, 128 biți

Nucleu criptografic în Rust / C

Post-cuantic: Kyber, Dilithium (SNDL  
termen lung)

JWT asimetric: RS256 / ES256

Sesiuni externalizate: Redis (scalabilitate  
orizontală)

HTTPS/TLS obligatoriu, semnare  
parametri publici

Jetoane proof-of-possession

Augmented PAKE: rezistență  
compromitere DB

Implementare schnorr interactive in faza  
de inserare credentiale in protocolul  
Oatub



# Concluzii

- Se poate folosi Schnorr intr-un protocol de tip Open Authorization
- O implementare ZKP de acest tip anuleaza o bresa a bazei de date, sau o exfiltrare a datelor
- Audit trafic: exclusiv valori matematice efemere în rețea
- Performanță intermediară față de PAKE consacrate
- Potential de optimizare Schnorr-EC-C-Lib: **9,51 ms**
- Limitări: viteza, verificare browser
- **Concluzie:** ZKP integrabil eficient în arhitectură web modernă, reducere semnificativă expunere credențiale

# Bibliografie SELECTIVĂ

---

- [9] S. Goldwasser, S. Micali, C. Rackoff, "The Knowledge Complexity of Interactive Proof Systems," SIAM J. Computing, 1989.
- [16] C.P. Schnorr, "Efficient Signature Generation by Smart Cards," J. Cryptology, 1991.
- .

VĂ MULȚUMESC  
PENTRU ATENȚIE!

---